The rationale for this spacing is that if you squint hard enough, when and rescue aren't really subordinate blocks and shouldn't be indented. In practice, like the two-space indenting, this lack of indentation tends to make Ruby code more compact on the screen than code from other languages.

If you use private as a keyword, typically methods below it are not indented, again because private is a method call and does not create a logical block:

```
class Sample
  def public_method
  end

  private

  def private_method
  end
end
```

As you've seen in the code samples in the book so far, Ruby code uses the end of a line to end statements, so you don't typically use a semicolon at the end of a line.

Ruby code style uses spaces around operators, around equals signs, and after commas, but does not uses spaces inside square brackets or parentheses. For example:

```
def do_some_things(x, y = "default")
  a = x + y
  b = [1, 2, 3]
end
```

There are a few exceptions:

- Range literals generally don't have spaces, as in 1..10
- When ! is used as a negation operator it doesn't have spaces, as in !foo
- The &. safe navigation operator doesn't get spaces so that it visually is just part of a method chain as in user&.name&.first.
- Exponent operators don't have spaces, as in x**2
- Rational and complex literal markers don't have spaces, as in 3/5r

Curly braces are a little more complicated. The most common style is to put spaces inside curly braces for block literals, but not for hash literals or string interpolation, with the idea being that you use the spacing to give an immediate visual clue whether the braces enclose a block or a hash:

```
def sample
  data = {a: 1, b: 2}
  data.each { |key, value| p "#{key}/#{value}" }
end
```

That said, you will see styles that put spaces inside hash literals as well.

As for block literals, the most common Ruby style is to use curly braces for single-line blocks and do/end for multiline blocks:

```
def another_sample
  [1, 2, 3].map { |x| x**2 }
  [1, 2, 3].map do |x|
    print "working on #{x}"
```